

گزارش پروژه معماری کامپیوتر دکتر جهانگیر

نادر احمدی - ۴۰۳۱۰۵۶۷۴

دانشگاه صنعتی شریف - ۱۴۰۵

توضیحات

برای انجام این پروژه از پردازنده M2 ساخت شرکت اپل استفاده شده است.

۱ بخش اول: استخراج اثر انگشت ریزمعماری با زمان سنجی

در این بخش قصد داریم موارد زیر را بررسی کنیم.

- اندازه سطوح حافظه نهان (L1 L2 L3)
- اندازه خط حافظه نهان (Cache Line Size)
- درجه شرکت پذیری (N-way Set-Associative)

۱.۱ اندازه سطوح حافظه نهان

در این قسمت سعی می‌کنیم با داده‌هایی با اندازه‌های 4 KB تا 64 MB، تاخیر سطوح مختلف حافظه نهان را بسنجیم. این اندازه‌ها در هر تکرار (Iteration) دو برابر می‌شوند تا بتوانیم مرزهای بین لایه‌های مختلف را به‌دقت تشخیص دهیم. ابتدا آرایه‌ای از گره‌ها (Node) ایجاد می‌شود که اندازه هر گره برابر با 128 بایت است. تعداد گره‌ها در هر گام از تست، از تقسیم حجم کل داده بر اندازه هر گره به دست می‌آید. علت انتخاب 128 بایت، پر کردن کامل یک خط نهان و جلوگیری از قرارگیری یک گره روی مرز دو بلوک مجزا (Misalignment) است. البته مقدار 64 بایت نیز مورد آزمایش قرار گرفت که به دلیل حفظ همترازی، نتایج کاملاً یکسانی حاصل شد.

اتصال بین گره‌ها بر اساس روش Pointer Chasing پیاده‌سازی شده است به این صورت که هر گره حاوی آدرس گره بعدی بوده و زنجیره‌ای متوالی را تشکیل می‌دهند. همچنین برای غیرفعال‌سازی پیش‌بین سخت‌افزاری (Hardware Prefetcher) و جلوگیری از کشف الگوی دسترسی توسط پردازنده، از الگوریتم Fisher-Yates Shuffle استفاده شده است. این الگوریتم با پیمایش از انتهای آرایه به سمت ابتدا، اشاره‌گر گره بعدی (next) را بین گره‌های مختلف جابه‌جا می‌کند.

```
void shuffle_nodes(struct Node* array, size_t count) {
    for (size_t i = 0; i < count; i++) {
        array[i].next = &array[(i + 1) % count];
    }
    srand((unsigned int)time(NULL));
    for (size_t i = count - 1; i > 0; i--) {
        size_t j = rand() % i;
        struct Node* temp = array[i].next;
        array[i].next = array[j].next;
        array[j].next = temp;
    }
}
```

شکل ۱: Shuffle Fisher-Yates

از آنجایی که حرکت در زنجیره بسیار سریع بوده و امکان اندازه‌گیری زمان اجرا ممکن نیست، در هر دور ۱۰ میلیون بار عمل حرکت در زنجیره را تکرار می‌کنیم تا زمان خروجی ملموس باشد. خروجی این مرحله به این صورت است.

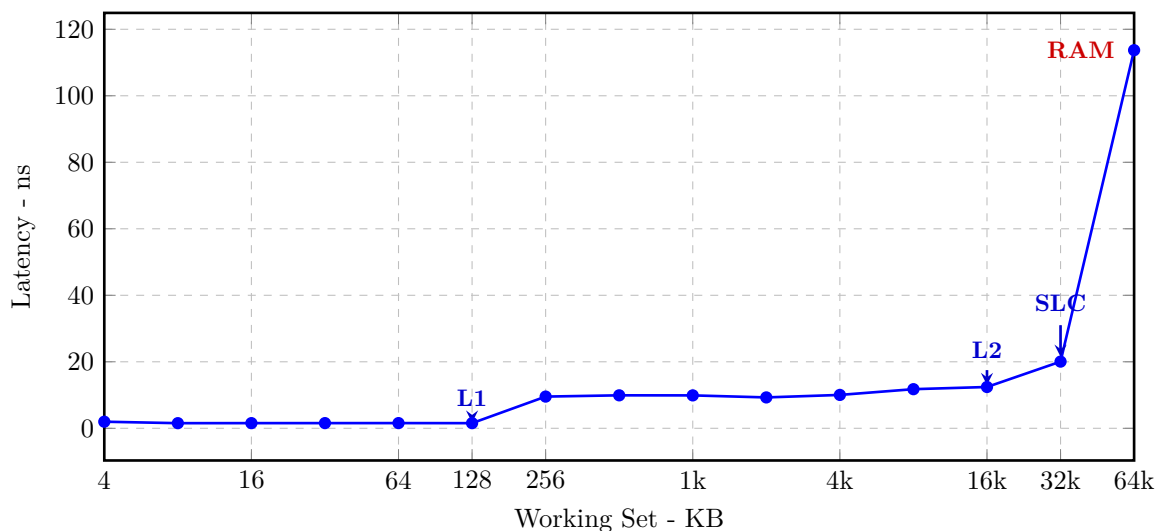
```

=== TEST 1: Extracting Cache Levels (Cache Sizes) ===
Working Set (KB)    Num Nodes    Latency (ns)
-----
4.00                32          1.980
8.00                64          1.538
16.00               128          1.551
32.00               256          1.555
64.00               512          1.562
128.00              1024          1.543
256.00              2048          9.534
512.00              4096          9.913
1024.00             8192          9.895
2048.00             16384         9.284
4096.00             32768         10.033
8192.00             65536         11.763
16384.00            131072        12.418
32768.00            262144        20.054
65536.00            524288        113.705
>> TEST 1 Total Execution Time: 2555.399 ms (2.555 s)

```

شکل ۲: آزمون اندازه سطوح حافظه

Test1: Cache Layers Size



نتایج

همان طور که دیده می شود تا ۱۲۸KB تاخیر واکنشی داده در حدود ۱.۵ نانوثانیه بوده است که این سطح در واقع همان **سطح اول - L1** حافظه نهان است. در ادامه از ۲۵۶KB تا ۱۶MB تاخیر در حدود ۹.۵ الی ۱۲ نانوثانیه بوده است که این همان حافظه نهان **سطح دوم - L2** می باشد. بعد از آن نیز جهش قابل توجهی دیده می شود که برای دسترسی به حافظه اصلی می باشد.

توجه: سری پردازنده های M2 از حافظه نهان لایه سه بهره مند نیستند ولی دارای SLC می باشند که بین بخش های مختلف تراشه به اشتراک گذاشته شده است محدوده این حافظه نهان بین ۱۶MB تا ۳۲MB ارزیابی می شود.

< حال باید خروجی ها را با مقادیر صحیح مقایسه کنیم. برای بدست آوردن مقادیر صحیح از دستور زیر استفاده شده است.

```
sysctl hw.perflevel0.l1d cachesize hw.perflevel0.l1i cachesize
hw.perflevel0.l2 cachesize
```

خروجی این دستور به این صورت است:

```
hw.perflevel0.l1d cachesize: 131072
hw.perflevel0.l1i cachesize: 196608
hw.perflevel0.l2 cachesize: 16777216
```

که تا مقدار زیادی با خروجی ما یکسان است.

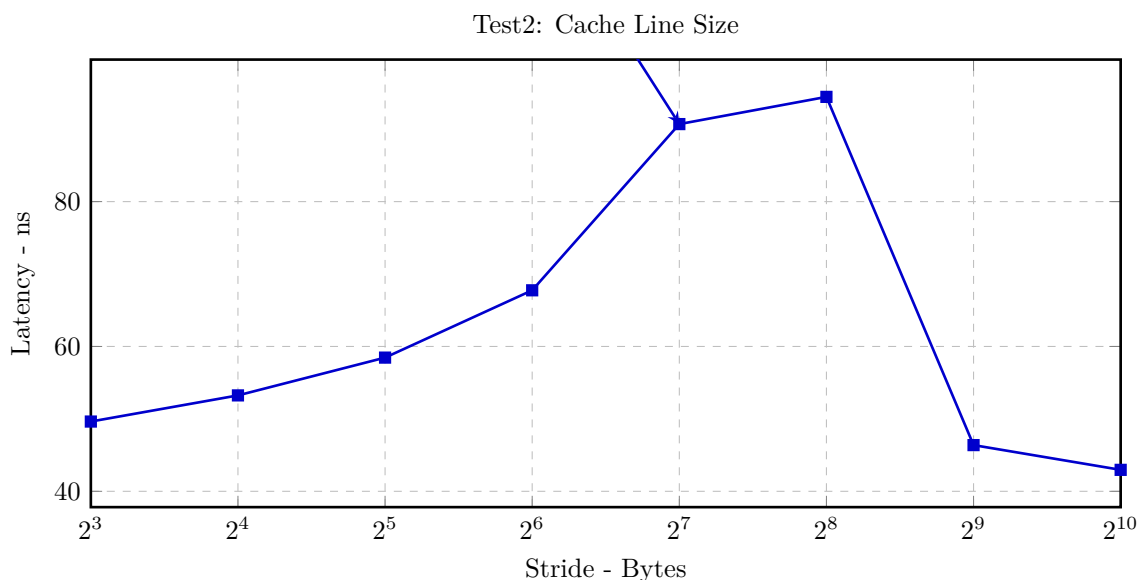
البته گاهی ممکن است مرز دقیق جهش تاخیر در خروجی های مختلف اندکی متفاوت باشد. علت این موضوع آن است که بخشی از ظرفیت حافظه نهان سطح اول ممکن است در حین اجرا، توسط برنامه های پس زمینه سیستم عامل اشغال شده باشد.

۲.۱ اندازه خط حافظه نهان (Cache Line Size)

در این بخش، هدف تعیین اندازه دقیق هر خط (بلوک) از حافظه نهان از طریق تغییر گام پیمایش (Stride) است. برای انجام این تست، آرایه‌ای از داده‌ها به حجم 32 MB تخصیص داده می‌شود تا کل داده خارج از حافظه نهان L2 و درون RAM قرار گیرد و تاخیر دسترسی به حافظه به‌وضوح قابل سنجش باشد. در هر مرحله، حجم کل داده بر مقدار Stride تقسیم می‌شود تا تعداد گره‌های زنجیره مشخص گردد. واضح است که با افزایش مقدار Stride، تعداد کل گره‌های قابل پیمایش در آن چرخه کاهش می‌یابد.

```
=== TEST 2: Extracting Cache Line Size ===
Stride (Bytes)      Latency (ns)
-----
8                   49.632
16                  53.243
32                  58.460
64                  67.753
128                 90.703
256                 94.460
512                 46.379
1024                42.963
>> TEST 2 Total Execution Time: 7673.655 ms (7.674 s)
```

شکل ۳: آزمون اندازه خطوط حافظه نهان



نتایج

همان‌طور که در نمودار شکل ۳ مشاهده می‌شود، در گام ۱۲۸ بایت یک جهش ناگهانی در تاخیر دسترسی رخ می‌دهد بنابراین می‌توان عدد ۱۲۸ بایت را عنوان اندازه خطوط حافظه نهان (Cache Line Size) نتیجه‌گیری گرفت. علت کاهش تاخیر در گام‌های بعدی (Stride‌های ۲۵۶ بایت به بالا) این علت است که با بزرگ‌تر شدن Stride، تعداد کل گره‌ها به‌شدت کاهش پیدا می‌کند در نتیجه تعداد دسترسی‌ها کم می‌شود. جهت بررسی و صحت‌سنجی این نتیجه، دستور زیر در محیط ترمینال اجرا شد:

```
sysctl hw.cachelinesize
```

خروجی این دستور به‌صورت زیر است:

```
hw.cachelinesize: 128
```

که نشان می‌دهد نتیجه آزمایش ما درست بوده است.

۳.۱ درجه شرکت پذیری

در این بخش هدف تعیین تعداد بلوک‌هایی (Ways) است که می‌توانند به‌طور هم‌زمان درون یک مجموعه (Set) مشخص از حافظه نهان قرار گیرند. برای این منظور، داده‌ها با فاصله ثابت 16 KB از یکدیگر انتخاب می‌شوند، این کار باعث می‌شود

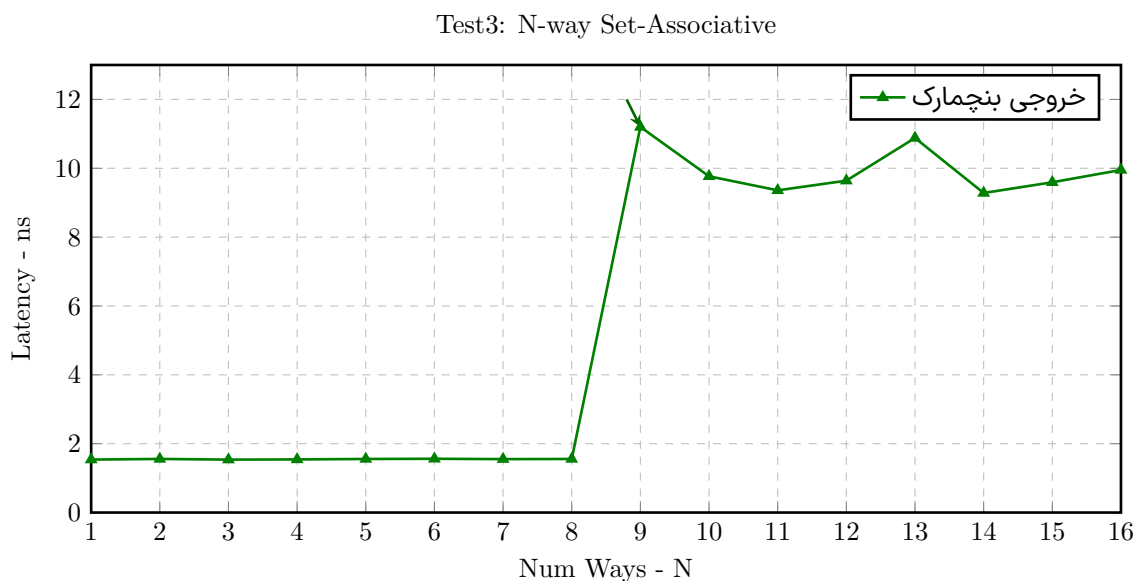
شود که مطمئن شویم داده متعلق به یک مجموعه هستند. سپس با افزایش گام به گام تعداد این داده‌ها از 1 تا 16، ظرفیت آن مجموعه مورد ارزیابی قرار می‌گیرد. تا زمانی که تعداد داده‌ها از درجه شرکت پذیری واقعی پردازنده فراتر نرفته باشد، تمامی آن‌ها درون همان مجموعه جا شده و تاخیر دسترسی در پایین‌ترین حد باقی می‌ماند. اما به محض پر شدن ظرفیت مجموعه، داده‌های جدید باعث رخ دادن خطای برخورد (Conflict Miss) و خروج داده‌های قبلی می‌شوند که این امر با یک جهش ناگهانی در تاخیر دسترسی، درجه دقیق شرکت پذیری را مشخص می‌سازد. خروجی این مرحله به این صورت است.

```

=== TEST 3: Estimating Cache Associativity ===
Num Ways (N)      Latency (ns)
-----
1      1.539
2      1.557
3      1.538
4      1.543
5      1.556
6      1.562
7      1.552
8      1.556
9      11.200
10     9.767
11     9.359
12     9.640
13     10.880
14     9.282
15     9.593
16     9.951
>> TEST 3 Total Execution Time: 920.951 ms (0.921 s)

```

شکل ۴: آزمون درجه شرکت پذیری



نتایج

همان طور که دیده می‌شود در درجه ۹ یک جهش ناگهانی در تاخیر داریم پس می‌توان نتیجه گرفت که درجه شرکت پذیری در پردازنده ما ۸ می‌باشد. با این حال خروجی دقیقی از این مقدار در اینترنت یافت نشد ولی طبق پاسخ LLM ها، درجه حافظه نهان سطح اول ۸ می‌باشد. پس نتیجه آزمایش درست می‌باشد.

۲ بخش دوم: خواندن شمارنده های سخت افزاری پردازنده

توضیحات

از آنجایی که ابزار های پیشنهاد شده در توضیحات پروژه برای سیستم عامل مک به صورت مستقیم موجود نبود، از ابزار **Instruments موجود در Xcode** استفاده شده است. این ابزار کمک می کند که عملکرد بخش های مختلف CPU را زیر نظر گرفت. برای رصد شمارنده های پردازنده کافیت برنامه اجرایی مورد نظر را در این ابزار انتخاب کرده و سپس با اضافه کردن Event ها به پارامتر های مورد نیاز دسترسی پیدا کرد. خروجی این ابزار به فرمت **Trace** می باشد که توسط خود ابزار Instruments قابل اجراست.

برای بررسی این بخش از برنامه نوشته شده در بخش اول استفاده شده است. پس از اجرای برنامه، خروجی های زیر بدست آمده است که در یک جدول آورده شده است.

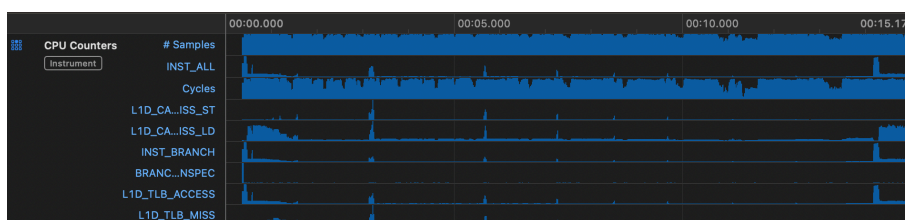
Table 1: Microarchitecture Metrics Extracted from Xcode Instruments for Process (Samples: 141,538)

Microarchitectural Metric	Hardware Event	Total Count	Calculated Rate / Ratio
Total Executed Instructions	INST_ALL	6,443,573,748	100.00%
Total CPU Clock Cycles	Cycles	25,523,723,774	–
Execution Efficiency (IPC)	INST_ALL / Cycles	–	0.2525 (CPI = 3.96)
L1 Data Cache Miss (Load)	L1D_CACHE_MISS_LD	295,389,509	–
L1 Data Cache Miss (Store)	L1D_CACHE_MISS_ST	23,571,344	–
Total L1 Data Cache Misses	L1D_CACHE_MISS (Total)	318,960,853	4.95% (of total instructions)
L1 Data TLB Accesses	L1D_TLB_ACCESS	3,533,290,451	–
L1 Data TLB Misses	L1D_TLB_MISS	163,779,190	4.64% (TLB Miss Rate)
Branch Instructions	INST_BRANCH	1,811,420,549	28.11% (of total instructions)
Branch Mispredictions	BRANCH_MISPRED_NONSPEC	18,873	0.0010% (Misprediction Rate)
Profiling Resolution	# Samples	141,538	≈ 10,011 Hz (Samples/sec)

و زمان اجرای توابع مختلف این برنامه در زیر آمده است.

Table 2: Function Execution Time Distribution (Time Profiler Breakdown)

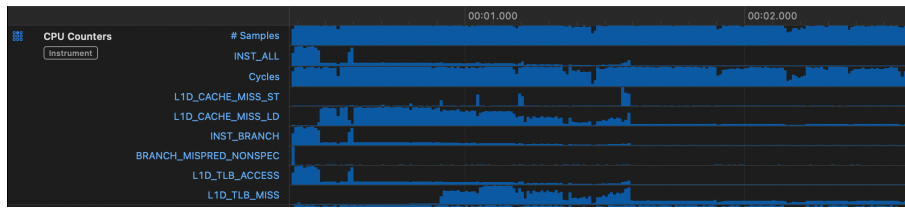
Benchmark Function	Total Time (Weight)	Self Time	% of Total Runtime
run_cache_line_test	10.52 s	10.34 s	74.4%
run_cache_size_test	2.67 s	2.63 s	18.9%
run_associativity_test	948.20 ms	948.10 ms	6.7%
Total Execution Time	14.14 s	–	100.0%



شکل ۵: خط زمانی اجرای برنامه در ابزار Instruments

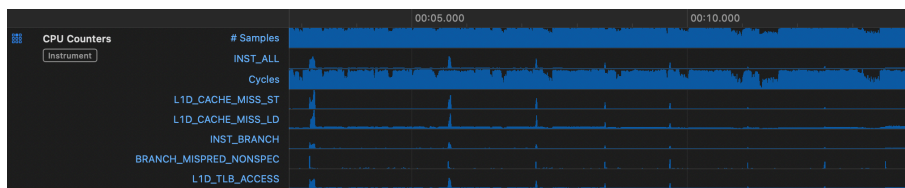
۱.۲ پیوند با بخش اول

حال بخش های مختلف این خط زمانی رو بررسی می کنیم.



شکل ۶: خط زمانی آزمون اندازه سطوح حافظه نهان

همان طور که دیده می شود در ابتدا تعداد miss های لایه ۱ حافظه نهان کم است ولی بعد از بزرگ شدن حجم داده ها، ناگهان تعداد خطاها زیاد می شود که همین وضعیت در بخش اول نیز دیده شده است. علاوه بر این موضوع دیده می شود که هر چه به پایان اجرای برنامه این بخش نزدیک می شویم تراکم و ارتفاع خطاها مجدد کاهش پیدا می کند و این به این علت است که در حجم های بالا مانند 32 MB, 16MB با ایجاد هر خطا تاخیری زیادی برای بازیابی داده صرف می شود که باعث مسدود شدن پایپلاین شده و در نتیجه تراکم miss ها کاهش پیدا کند.



شکل ۷: خط زمانی آزمون اندازه خط حافظه نهان

همان طور که دیده می شود، در نقاطی از اجرا پیک‌هایی رخ داده است و در بقیه نواحی میزان خطای حافظه نهان (Miss) مقداری تقریباً یکنواخت دارد. این پیک‌ها در واقع نشان‌دهنده مراحل آماده‌سازی و درهم‌سازی (Shuffle) داده‌ها در هر دور از اجرای حلقه می‌باشند. از آنجا که در مراحل اولیه (Stride های کوچک‌تر) تعداد گره‌ها بیشتر است، شدت و تراکم پیک‌ها نیز بیشتر دیده می‌شود. در بقیه نقاط نیز خطایی نسبتاً یکنواخت مشاهده می‌شود، علت آن است که به دلیل بالا بودن تاخیر بازیابی داده از RAM (چون داده اولیه دارای 32MB است)، شدت و تعداد خطا در واحد زمان کاهش یافته و ثابت می‌ماند. نکته دیگری که دیده می‌شود، بیشتر بودن فاصله پیک‌ها در ابتدای اجرا نسبت به انتها است که علت آن، همان بار محاسباتی (Overhead) بالاتر و همچنین بیشتر بودن تعداد گره‌ها در چرخه های اولیه اجرا می باشد.



شکل ۸: خط زمانی آزمون درجه شرکت پذیری

در ابتدا و در درجه های کمتر به علت اینکه همه داده ها درون یک مجموعه جا می شوند، تعداد miss ها بسیار کم می باشد ولی با افزایش تعداد این درجات تمام داده ها درون یک مجموعه جا نشده و مجبور به اجرای سیاست های جایگزینی می شود و در نتیجه تعداد miss ها شدت گرفته است.

۳ بخش سوم: تحلیل و بهبود یک پدیده عملکردی

پدیده مورد بررسی: پیمایش سطری در برابر ستونی

۱.۳ فرضیه

از آنجایی که داده‌های ماتریس در حافظه به صورت پشت سر هم قرار گرفتن سطرها ذخیره می‌شوند، انتظار داریم که در حالت پیمایش سطری، بنا بر اصل مجاورت مکانی، خطاها و تاخیر کمتری داشته باشیم.

۲.۳ آزمایش و نتیجه‌گیری

آزمایشی برای بررسی این پدیده طراحی شده است. در این آزمایش یک ماتریس به اندازه 4096×4096 تعریف شده که درون هر خانه داده‌ای به مقدار ۴ بایت قرار گرفته است. در حالت پیمایش سطری، محاسبه `matrix[i * SIZE + j]` را به صورت `i + j` و در پیمایش ستونی این کار را به صورت ستونی انجام می‌دهیم. قطعه کدی که برای پیمایش استفاده شده در زیر آمده است:

```
for (int j = 0; j < SIZE; j++) {
    for (int i = 0; i < SIZE; i++) {
        matrix[i * SIZE + j] += i + j;
    }
}
```

شکل ۹: کد بخش پیمایش ستونی

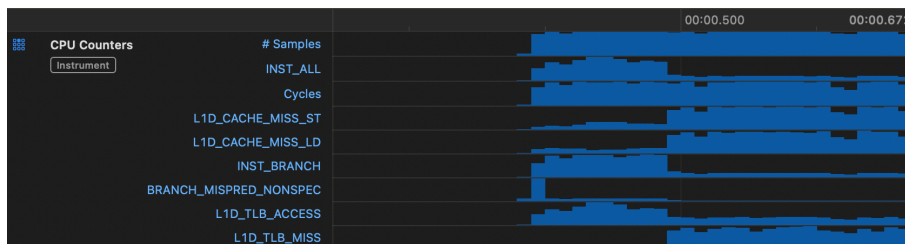
حال نتایج آزمایش درون CommandLine و Instruments در زیر آمده است:

```
=== TEST 1: Row-Major Matrix Traversal ===
Row-Major Traversal Time: 27.483 ms
>> TEST 1 Total Execution Time: 64.351 ms

=== TEST 2: Column-Major Matrix Traversal ===
Column-Major Traversal Time: 168.776 ms
>> TEST 2 Total Execution Time: 199.452 ms

=====
>> TOTAL PROGRAM EXECUTION TIME: 263.810 ms (0.264 s)
=====
```

شکل ۱۰: خروجی درون CommandLine



شکل ۱۱: خروجی درون ابزار Instruments

همان‌طور که دیده می‌شود، زمان اجرا و تعداد خطاهای دسترسی به حافظه نهان L1 در حالت ستونی بسیار بیشتر از حالت پیمایش سطری می‌باشد.

علت این مسئله به همان اصل مجاورت در حافظه نهان باز می‌گردد. هنگامی که حافظه نهان به بخشی از حافظه دسترسی پیدا می‌کند، برای افزایش نرخ موفقیت داده‌های همسایه داده‌ی مورد رجوع را نیز درون خود ذخیره می‌کند زیرا احتمال می‌رود این داده‌ها به زودی مورد نیاز واقع شوند. حال در پیمایش سطری به علت ماهیت نحوه ذخیره‌سازی ماتریس‌ها درون حافظه، با خواندن یکی از داده‌ها، داده‌های بعدی درون آن سطر نیز وارد حافظه نهان می‌شوند و در نتیجه نرخ موفقیت (Hit Rate) افزایش پیدا می‌کند. از طرفی در حالت ستونی، به علت این که داده‌ها درون سطریهای مختلفی قرار دارند، خطای دسترسی (Miss Rate) افزایش یافته و مطابق نمودار زمانی، تراکم خطاها افزایش می‌یابد.

۳.۳ رد فرضیه‌های رقیب: خطاهای پرشی

شاید این تصور به وجود آید که در حالت ستونی پیش‌بینی پرش‌ها دشوارتر بوده و در نتیجه تاخیر تا این حد افزایش یافته است. اما با مشاهده خروجی شمارنده‌های پردازنده متوجه می‌شویم که تعداد Branch Mispredictionها فقط در

Table 3: Hardware Counters - before optimzing

Metric	Samples	INST_ALL	Cycles	MISS_ST	MISS_LD	INST_BRANCH	MISPRED_NONSPEC	TLB_ACCESS	TLB_MISS
Total	2747	1,109,406,473	516,911,846	20,379,099	20,228,561	260,724,223	28,546	690,529,492	128,591,428

ابتدای پیمایش سطری وجود داشته و سپس تا انتهای اجرا در یک مقدار بسیار کم ثابت می ماند. پس می توان نتیجه گرفت که این عامل تأثیر چندانی روی تأخیر به وجود آمده ندارد.

۴.۳ حالت بهینه

در این حالت برای کاهش تأخیر و خطاهای حافظه نهان در پیمایش ستونی، از روش پیمایش در پنجره های کوچک تر (Cache Tiling) بهره گرفته ایم. با تعریف زیرماتریس های کوچک تر به اندازه 32×32 ، پیمایش را درون این ماتریس های کوچک انجام می دهیم. در این صورت به علت کم بودن حجم این ماتریس ها (4 KB)، داده ها به طور کامل درون حافظه نهان قرار گرفته و تأخیر کاهش پیدا می کند. ارقام حاصل از حالت بهینه یافته در زیر آمده است:

Table 4: Hardware Counters - Optimized

Metric	Samples	INST_ALL	Cycles	MISS_ST	MISS_LD	INST_BRANCH	MISPRED_NONSPEC	TLB_ACCESS	TLB_MISS
Total	2208	1,157,513,145	418,807,471	10,803,379	25,678,832	265,005,114	42,767	661,948,878	4,542,589

حال مقایسه حالت بهینه یافته و حالت اولیه:

Table 5: Hardware Counters Comparison

Hardware Metric	Original (Unoptimized)	Optimized (Tiled)	Improvement / Change
Execution Time (ms)	271.60	217.20	-20.0% (Faster)
Samples	2,747	2,208	-19.6%
Total INST_ALL	1,109,406,473	1,157,513,145	+4.3% (Overhead)
Total Cycles	516,911,846	418,807,471	-19.0% (Faster)
L1D_CACHE_MISS_ST	20,379,099	10,803,379	-47.0% (Reduced)
L1D_CACHE_MISS_LD	20,228,561	25,678,832	+26.9%
INST_BRANCH	260,724,223	265,005,114	+1.6%
BRANCH_MISPRED_NONSPEC	28,546	42,767	+14,221 (< 0.01% total)
L1D_TLB_ACCESS	690,529,492	661,948,878	-4.1%
L1D_TLB_MISS	128,591,428	4,542,589	-96.5% (Massive Drop)

۵.۳ پیوند میان بخش ها

طبق داده های بخش اول متوجه شدیم که اندازه خط حافظه نهان (Cache Line) برابر با ۱۲۸ بایت می باشد بنابراین با خواندن هر داده، ۱۲۸ بایت داده وارد حافظه نهان می شود. حال در حالت پیمایش ستونی بهینه نشده که اندازه هر ستون ۴۰۹۶ خانه است، پس از پیمایش کامل یک ستون، حجمی معادل $4096 \times 128 = 512$ KB داده وارد حافظه نهان می شود. از آنجایی که با توجه به نتایج بخش اول، ظرفیت لایه اول حافظه نهان (L1D) این پردازنده ۱۲۸ کیلوبایت است، پردازنده مجبور می شود خطوط Cache مربوط به سطرهاى اولیه را پیش از استفاده مجدد از حافظه نهان خارج کند که همین امر باعث افزایش شدید خطای دسترسی (Miss) می گردد.